

Supervision

Student Supervisor	
Posted at	05/15/2026
Main activities	During this month, the internship work focused on three main technical streams: strengthening automated testing, advancing the Airtable-to-PostgreSQL migration scaffold, and formalizing the separation between development and production environments. The first activity was the implementation of a more complete API testing strategy. The backend now includes a structured Vitest setup with: unit tests for API/database behavior, smoke/contract tests for the database layer, support for running the same smoke suite against two backends (airtable and postgres). To support realistic test execution, a dedicated Docker Compose test stack was introduced. This stack orchestrates: a Node test container, a PostgreSQL test container, an Airtable mock HTTP service (FastAPI wrapper over airtablemock). In parallel, CI was upgraded to run tests as a first-class pipeline stage. GitLab CI now executes: test:unit, test:smoke:airtable, test:smoke:postgres. The second major activity was progressing the Airtable-to-PostgreSQL migration work. A dual-database backend path is now implemented through the DATABASE selector (airtable/postgres) with shared API behavior. Database modules were progressively adapted to support PostgreSQL reads/writes while preserving Airtable-compatible payload structures. At infrastructure/tooling level, migration support was expanded with: Knex + pg integration, a bootstrap migration generated from the Airtable schema contract, replication and validation scripts (replicate:airtable, validate:airtable) producing JSON reports, updated documentation for schema migration and data verification flow. The third activity was environment segmentation between prod and dev. Aurora PostgreSQL is currently enabled only for dev, while production remains conservative. The setup also separates ownership/reuse of shared simulation storage resources, and deployment pipelines continue to map to dedicated environment files.
Achievement(s)	This month marked a major transition from preparation to implementation for the migration project. A robust API testing architecture was introduced, combining unit, smoke, and contract tests with backend switching between Airtable and PostgreSQL. To support this, a fully Dockerized test environment was created, including a custom Airtable mock service and a PostgreSQL test

database for reproducible local and CI execution.

GitLab CI was also extended with a dedicated testing stage running automated unit and smoke tests, strengthening quality validation before deployment.

Significant progress was made on the migration scaffold itself, including runtime backend selection, PostgreSQL integration across core modules, Knex migrations, and Airtable-to-Aurora replication and validation scripts with machine-readable reporting.

Finally, the Terraform infrastructure was improved with a clearer separation between dev and prod environments, consistency checks, dev-only Aurora enablement, and a cleaner DNS and shared storage strategy.

Overall, this month established a solid, testable, and automatable foundation for the migration effort.

Troubleshooting(s)

Several technical challenges were addressed during implementation.

One key challenge was maintaining consistent API behavior across both Airtable and PostgreSQL while adapting existing Airtable-oriented logic to support the new backend without regressions.

Another challenge was designing backend-agnostic smoke tests, requiring shared test helpers capable of handling both Airtable mock flows and PostgreSQL setup/cleanup in Docker and CI environments.

Finally, infrastructure consistency between dev and prod environments required stricter Terraform conventions, controlled feature rollout, and careful management of shared resources to prevent unintended production impact.

Next step(s)

The next phase will focus on completing the functional decoupling from Airtable and consolidating core platform services on the new architecture.

Implement User Sign-In Process and Authentication Flow

A dedicated authentication flow must be implemented at application level (sign-in, session handling, and permission checks), because part of the previous user access logic was implicitly tied to Airtable-based structures and workflows.

This step is required to make authentication independent, reliable, and fully controlled by the backend platform.

Replace Airtable Automations with backend-native services

The platform must progressively replace Airtable automations that are still providing operational logic outside the codebase.

A priority example is the Slack Messaging Integration for System Notifications, which should be migrated to application-managed notification workflows to ensure maintainability, observability, and environment consistency.

**Intermediate report
(only one per month)**
